

PROJECT REPORT

Enterprise Employee Assistant

A plain-English guide to what the system does, how it is built, and why each piece of technology was chosen over the alternatives.

Multi-agent HR & IT support assistant · Amazon Bedrock AgentCore · Strands Agents
Prepared for internal review

1. What is this project?

In one sentence: it's a chat assistant for employees that can answer HR questions (like leave balance) and IT questions (like device or ticket status), safely, using each employee's real identity — without a human ever needing to manually pull up company records.

Every company has two departments that get asked the same handful of questions over and over: HR ("how many leave days do I have left?") and IT ("what's the status of my ticket?", "my laptop won't connect to Wi-Fi"). Normally, an employee either digs through an internal portal or messages a person on the HR/IT team and waits. This project replaces that wait with a chat assistant that can look up the real answer immediately, because it is wired directly into the same databases and systems the HR/IT teams already use.

What it can actually do today

- **HR questions:** "What's my leave balance?", "Show my employee details", "What are the company holidays?", questions about HR policies and benefits.
- **IT questions:** "What's the status of my laptop?", "Create a ticket for my Wi-Fi issue", "Show me ticket INC-1001", "Close ticket INC-1002", IT policy questions, and even looking things up on the open internet (e.g. a printer driver page) when the answer isn't in company systems.
- **Remembers context:** it keeps track of the conversation and some long-term facts about the employee across sessions, so you don't have to repeat yourself.
- **Knows who is asking:** answers are always scoped to the real, logged-in employee — an employee can't ask for someone else's leave balance and get it, because the system checks who is actually asking before it hands back any data.
- **Never closes a ticket on its own:** a sensitive action like closing an IT ticket is held for a human admin to approve first — the assistant can suggest it, but a person has to say yes.

Who is involved

ROLE	WHAT THEY GET
Employee	A chat assistant that answers their own HR/IT questions using their real employee record.
IT Admin	Everything an employee gets, plus the ability to look up any employee's device/ticket and see organization-wide lists.
The business	Fewer repetitive tickets/questions landing on human HR & IT staff, answered instantly and consistently, 24/7.

2. How it works, in plain terms

Think of it like a small company inside the software: one "manager" and two "specialists."

- 1 **You log in and ask a question** in the chat app. Your identity is proven using your normal company login (through Amazon Cognito — more on that below), not typed in as text, so it can't be faked.
- 2 **The "Manager" (Orchestration Agent)** reads your question and decides: is this an HR question or an IT question? It never tries to answer enterprise questions itself — it just routes.
- 3 **The right "Specialist" takes over.** The HR Agent handles leave/employee/benefits questions. The IT Agent handles devices, tickets, IT policy, and can even browse the public internet for things like driver downloads or vendor status pages.
- 4 **The specialist looks up the real answer** — from the company database, from an internal policy document library, or (for IT) an actual live webpage — instead of guessing.
- 5 **Sensitive actions pause for a human.** If you ask to close a ticket, the system logs the request as "pending" and waits for an IT admin to approve it before anything actually changes.
- 6 **You get a plain-English answer**, and the conversation is remembered for next time.

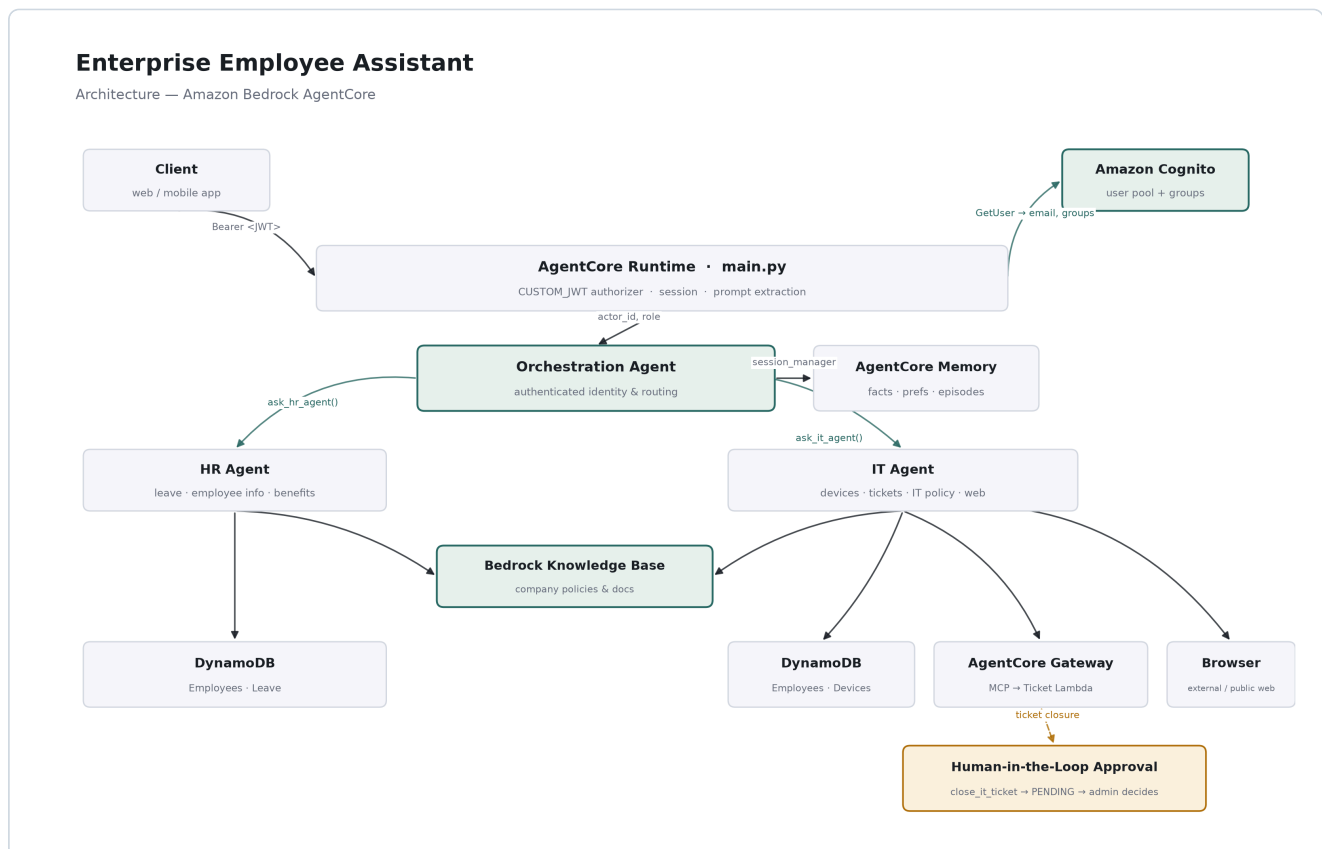


Figure 1 — the full request flow, component by component (see Section 4 for what each box means).

Why a "manager + specialists" design, instead of one big AI?

We could have built one single AI agent that tries to do everything. We didn't, for three practical reasons:

- **Focus improves accuracy.** An agent with 5 IT-specific tools and a tightly-scoped job description makes fewer mistakes than one agent juggling 15+ tools across two unrelated departments.
- **Permissions are easier to reason about.** The IT Agent literally cannot see HR tools, and vice versa — there's no code path for it to even try.
- **It's easier to extend.** Adding a third department (say, Finance) later means adding a third specialist, not rewriting one giant prompt that already does two jobs.

The specific pattern: "Agents-as-Tools," not a Graph or a Swarm

The AI framework we use (Strands) actually offers three different ways to connect multiple agents together. We picked the simplest one on purpose. Here's the difference in plain terms:

PATTERN	WHAT IT MEANS	USED HERE?
Agents-as-Tools	One "manager" agent, with each specialist wrapped up so the manager can call it exactly like a tool — ask it something, get an answer back.	✅ Yes
Graph	A fixed, multi-step pipeline between several agents — e.g. "draft this, then have a second agent critique it, then a third agent revise it," with the pipeline able to branch or repeat steps.	No
Swarm	A group of equal, independent agents that can hand a task off to each other on their own judgment, with no fixed manager in charge.	No

Why Agents-as-Tools won:

- **The decision is a single fork, not a pipeline.** Every message needs exactly one decision made once — "is this HR or IT?" — then it's handed off and answered. There's no multi-step chain (draft → review → revise) that would justify a Graph's extra machinery.
- **Letting agents freely hand off to each other (Swarm) is a security risk here, not a feature.** A Swarm's whole point is that any agent can decide mid-task to pass control to another agent. In this project, the IT Agent must never be able to reach into HR's tools (or the reverse) without going back through the identity-aware manager first — a Swarm would blur that boundary by design.
- **The employee's identity has to be locked in before a specialist agent is even created** — not passed around as a message. Each specialist is built fresh for every request with the real, verified employee ID already sealed inside it, so the AI itself never gets a chance to see or change whose identity it's acting as. Graph and Swarm are both built around agents exchanging messages/state with each other, which is exactly the channel this design keeps identity out of.
- **It's the simplest tool that does the job.** With only two specialists and a one-level decision, a full pipeline/graph engine would add complexity with no real benefit. Adding a third department later is just one more tool definition, not new pipeline wiring.

3. The technology stack, at a glance

LAYER	TECHNOLOGY	JOB IT DOES HERE
AI agent framework	Strands Agents SDK	Defines each agent's brain: its instructions, its tools, and how it talks to the language model.
AI hosting platform	Amazon Bedrock AgentCore	Runs the agents in the cloud, handles authentication, memory, tool access, and browsing.
Language model	Amazon Nova 2 Lite (via Bedrock)	The actual "brain" that reads questions and writes answers.
Content safety	Amazon Bedrock Guardrails	A filter that blocks unsafe, harmful, or manipulative inputs/outputs.
Login & identity	Amazon Cognito	Proves who the employee actually is and what role they hold.
Database	Amazon DynamoDB	Stores employee records, devices, leave balances, tickets, approvals, conversations.
Company knowledge search	Amazon Bedrock Knowledge Base	Lets agents search internal policy documents in plain English.
Tool connector	AgentCore Gateway (MCP protocol)	A secure bridge that lets an agent call an external backend (a ticketing Lambda) as if it were a local tool.
Serverless backend	AWS Lambda	Runs the ticket-detail lookup behind the Gateway without needing a server to maintain.
Conversation memory	AgentCore Memory	Remembers facts, preferences, and conversation history across sessions.
Web browsing	AgentCore Browser	Lets the IT Agent visit real external webpages (vendor docs, status pages) when asked.
Infrastructure as code	AWS CDK + AgentCore CLI	Deploys and manages all of the above from config files instead of manual clicking in a console.

Why AWS end-to-end? Every piece above — Cognito, DynamoDB, Bedrock, AgentCore, Lambda — lives in the same AWS account and region. That means one login, one bill, one security model, and no extra network hops between "the AI part" and "the data part." Mixing providers (e.g. OpenAI for the model, Auth0 for login, MongoDB Atlas for data) would work too, but it adds more moving parts, more vendor relationships, and more places for a security gap to hide.

4. Each technology, explained — what it is, why we used it, and the alternatives

Strands Agents SDK AGENT FRAMEWORK

What it is: A toolkit (from AWS) for building an AI "agent" — an AI that doesn't just chat, but can be given a job description and a set of tools (functions) it's allowed to call, and then decides for itself which tool to use for a given request.

Why we chose it: It plugs natively into Amazon Bedrock and AgentCore, so there's no glue code needed to make the model, the memory, and the tools talk to each other. Defining a tool is as simple as writing a normal Python function and adding a `@tool` label to it.

Alternatives: LangChain / LangGraph (more general-purpose, more setup, larger community), OpenAI's Assistants API / Agents SDK (ties you to OpenAI's models), Google's ADK (Gemini-only), Microsoft's Autogen (multi-agent focus, heavier framework). We picked Strands because our whole stack is already AWS-based, and Strands is purpose-built to run on AgentCore with minimal extra wiring.

Amazon Bedrock AgentCore AGENT HOSTING PLATFORM

What it is: A managed cloud service that runs your AI agent for you — like a web host, but purpose-built for AI agents. It takes care of scaling, authentication at the door, giving the agent a memory, letting it browse the web safely in a sandbox, and connecting it to outside tools.

Why we chose it: Building all of that ourselves (a server to run the agent, a login-checking layer, a memory database, a safe sandboxed browser) would be weeks of extra engineering. AgentCore gives us all of it as configuration, not code.

Alternatives: Self-hosting the agent on a plain server (e.g. an EC2 instance or a container on Kubernetes) running LangChain/Strands ourselves, using a different managed agent platform (e.g. Azure AI Foundry, Google Vertex AI Agent Builder), or a hosted-agent startup product. We chose AgentCore because it's the first-party AWS option, so it integrates directly with Cognito, DynamoDB, and Bedrock without any custom bridging code.

Amazon Nova 2 Lite LANGUAGE MODEL

What it is: The actual large language model — the part that reads a question in English and writes back an answer in English. "Lite" means it's a smaller, faster, cheaper version of Amazon's Nova model family, well suited to short, well-defined support questions rather than long creative writing.

Why we chose it: It's fast and inexpensive per request (important because every employee message triggers at least one model call, sometimes several across the two specialist agents), and it's accessed through Bedrock, so it comes with Guardrails and usage/observability built in for free.

How it's called: Through Bedrock's **Converse API** — the standard, unified way Bedrock lets an application send a conversation and a list of available tools to any supported model, and stream the reply back. This is the same interface regardless of which model sits behind it, which is part of why swapping Nova for Claude or another Bedrock model later would be a small change, not a rewrite.

Alternatives: Anthropic Claude, OpenAI GPT-4/GPT-5, Google Gemini — all of which are also reachable through Bedrock (Claude, and via the Strands framework, others directly). The code already has a commented-out path for Google Gemini, showing the model choice was deliberately kept swappable. Nova 2 Lite was picked as the default for its cost/speed profile on a support-chat workload; nothing about the architecture locks the project into it — swapping the model is a one-line change in `model/load.py`.

Amazon Bedrock Guardrails SAFETY FILTER

What it is: A rules-based safety net that sits between the user and the model. It checks messages (in both directions) for things like violent content, hate speech, or attempts to manipulate the AI into ignoring its instructions ("prompt injection").

Why we chose it: Since the model is already hosted on Bedrock, turning on a Guardrail is a configuration setting, not a separate product to integrate. It gives a second layer of defense on top of the instructions we already wrote into each agent telling it to refuse unsafe requests.

Alternatives: OpenAI's Moderation API, a self-built keyword/regex filter, or a third-party safety product like Lakera or Azure AI Content Safety. Bedrock Guardrails won because it requires zero extra network calls or vendor accounts — it's already attached to the model call we're making.

Amazon Cognito LOGIN & IDENTITY

What it is: A login system. It manages user accounts, passwords, group membership (e.g. "this person is in the ITAdmin group"), and issues a signed digital ticket (called a JWT, or "JSON Web Token") that proves who someone is for a limited time after they log in.

Why we chose it: Every request into the assistant carries this token, and the system checks it twice: once automatically at the front door (AgentCore's authorizer), and once again in our own code, to look up the caller's email and group membership. This is what makes it impossible for someone to simply type "I am an admin" in chat and be believed — the role comes from a cryptographically signed token, not from the conversation.

Alternatives: Auth0, Okta, Firebase Authentication, or a hand-rolled username/password system. We chose Cognito because, again, it's native to AWS and to AgentCore's built-in "CUSTOM_JWT" authorizer — no separate vendor, no extra monthly bill, and it already existed for the company for other tools.

Amazon DynamoDB DATABASE

What it is: A database for storing structured records — think of it as a set of spreadsheets that a program can read and write to extremely quickly. It stores employee profiles, device records, leave balances, IT tickets, pending approvals, and conversation history.

Why we chose it: Our data is simple "lookup by ID" data (find the record for employee EMP001, find ticket INC-1001) rather than complex relationships needing joins across many tables. DynamoDB is built exactly for that pattern, is extremely fast at it, and requires no database server to patch, size, or maintain — AWS runs it for us.

Alternatives: PostgreSQL or MySQL (traditional relational databases — more powerful for complex queries, but need a server to manage and are slower for simple key lookups at scale), MongoDB (similar "flexible record" idea to DynamoDB, but not a managed AWS-native service). DynamoDB won because our access patterns are simple, our stack is already on AWS, and "serverless" means no database maintenance work for the team.

Amazon Bedrock Knowledge Base DOCUMENT SEARCH

What it is: A system that takes a folder of documents (say, the company's PDF policy handbooks) and lets an AI search them using plain English questions, instead of needing exact keyword matches. This general technique is often called "RAG" (Retrieval-Augmented Generation).

Why we chose it: It means HR and IT policy answers come from the company's actual written policy — not from the AI's general internet-trained knowledge, which could be wrong or made-up for something company-specific. Both agents share the same Knowledge Base.

Alternatives: Building a custom search index with a vector database like Pinecone, Weaviate, or OpenSearch, or a simpler keyword search (like Elasticsearch). We chose the managed Bedrock option because it needs no separate database to run and upkeep, and it plugs straight into the same Bedrock account as the model and Guardrails.

AgentCore Gateway (MCP) + AWS Lambda TOOL CONNECTOR + BACKEND

What it is: "MCP" (Model Context Protocol) is an emerging open standard that lets an AI agent call an outside system's tools in a consistent way, regardless of who built that system. Our Gateway exposes one such outside tool — ticket-detail lookup — which is actually implemented as a small serverless function (a "Lambda") maintained by the ticketing system.

Why we chose it: It cleanly separates "things our own agent code does directly" from "things a different backend system owns." The Gateway also re-checks the caller's identity token before forwarding the call, so authorization isn't just trusted blindly.

Alternatives: Calling a plain REST API directly from the agent code, or a message queue (like SQS) for asynchronous ticket actions. MCP was chosen because it is becoming the standard way AI agents talk to external tools industry-wide, meaning this Gateway could later expose many more outside systems (not just tickets) without changing how the agent calls them.

AgentCore Memory CONVERSATION MEMORY

What it is: A managed memory store that keeps track of four kinds of things per employee: short-term facts, stated preferences, a running summary of long conversations, and "episodes" (notable past interactions) — so the assistant doesn't forget everything the moment a chat window closes.

Why we chose it: Building "AI memory" correctly (deciding what's worth remembering, summarizing old conversations so they don't overflow the model, separating one user's memory from another's) is a genuinely hard problem. AgentCore Memory does this automatically and keeps every employee's memory private to them.

Alternatives: Storing raw chat logs in DynamoDB ourselves and manually feeding old messages back to the model (cheaper, but loses the automatic summarization and "fact extraction" behavior), or a third-party memory product like Zep or Mem0. We used AgentCore Memory because it required no extra integration work — it's a config setting on the same platform.

AgentCore Browser WEB BROWSING TOOL

What it is: A safe, sandboxed web browser that the IT Agent can control — it can visit a real website, read the page, and pull out information, similar to a human opening a new browser tab.

Why we chose it: Some IT questions genuinely need the live internet — "is this vendor's status page reporting an outage?", "where's the driver download for this model?" — information that changes daily and can't live in our own Knowledge Base. Running this in AgentCore's sandbox means the browser session is isolated from our internal systems, so it can never be tricked into reaching internal data.

Alternatives: A general web-search API (e.g. Bing/Google Search API) that returns snippets instead of full pages, or a self-hosted headless browser (Puppeteer/ Playwright) that we'd have to secure and sandbox ourselves. AgentCore Browser was chosen because the sandboxing and safety boundary come built-in.

AWS CDK + AgentCore CLI INFRASTRUCTURE AS CODE

What it is: Instead of an engineer manually clicking through the AWS website to create each database table, Lambda function, and permission, the entire setup is described in configuration files (`agentcore.json`) and code (CDK), and a single command (`agentcore deploy`) builds or updates everything to match.

Why we chose it: It makes the whole system reproducible — anyone can recreate the exact same setup in a new AWS account — and it makes changes auditable, since they show up as file changes rather than untracked manual clicks.

Alternatives: Manual setup through the AWS Console (fast to start, but not repeatable or reviewable), Terraform (a very popular alternative infra-as-code tool, but not AWS-specific and would need custom modules for AgentCore's newer resources). We used the AgentCore CLI + CDK combination because it has first-party, ready-made support for every AgentCore resource this project uses.

5. How we keep this safe

An assistant that can read employee data and touch IT tickets has to be trustworthy by design, not just by good intentions in the prompt. Here's how that's enforced:

1. Identity comes from the login system, never from the chat text

When you type "I am an admin" or "show me employee EMP002's device," the system does not simply believe the sentence. Your real identity and role are resolved once, right at the start of the request, from your signed Cognito login token — and every specialist agent is explicitly instructed to ignore any claim about identity that shows up inside the conversation itself.

2. The AI is never the final word on "yes" or "no"

Every database tool independently re-checks whether the calling employee is allowed to see or change that specific record. Even if the AI model were somehow confused or tricked, the backend tools apply the real rule (e.g. "you can only see your own leave balance") regardless of what the model asked for.

3. Sensitive actions need a human "yes"

Closing an IT ticket doesn't happen instantly. It's written to an approvals table as "pending," and only a human IT admin's explicit approval actually executes the change. This pattern is called **human-in-the-loop**, and it exists specifically so that an AI mistake — or a cleverly worded request — can never silently close something real.

4. Two layers of authentication

Layer one: the cloud platform itself rejects any request without a valid, signed login token before our code even runs. Layer two: our own code separately asks the login system "who exactly is this token for, and what group are they in?" — so even a technically-valid-but-wrong-purpose token wouldn't be enough.

5. Content safety on both sides of the conversation

The Guardrail checks both what the user sends in and what the AI sends back, screening for things like attempts to manipulate the AI into breaking its own rules, or generating harmful content.

6. Why we built it this way

DESIGN DECISION	REASONING
Multi-agent (Orchestrator + HR + IT) instead of one agent	Smaller, focused jobs are more accurate and easier to secure than one agent doing everything.
Fully managed AWS services instead of self-hosted infrastructure	No servers to patch or scale manually; the team can focus on the agent's behavior, not on keeping infrastructure alive.
Human-in-the-loop for ticket closure	AI assistance should speed up routine lookups, not make irreversible changes unsupervised.
Real identity tokens instead of trusting chat text	Removes an entire category of "convince the AI you're someone else" attacks by design.
Backend tools re-check authorization independently	Defense in depth — a mistake in the AI's reasoning still can't leak data it shouldn't.
Everything on one cloud provider (AWS)	One security model, one bill, native integration between every piece, fewer places for a gap to appear between systems.

7. Quick glossary

TERM	PLAIN-ENGLISH MEANING
Agent	An AI that can use tools and make decisions about which tool to use, not just chat.
LLM (Large Language Model)	The AI model that reads and writes natural-language text — the "brain."
Orchestration	Deciding which specialist agent should handle a given request.
RAG (Retrieval-Augmented Generation)	Letting an AI search real documents for facts instead of relying only on what it was trained on.
JWT (JSON Web Token)	A tamper-proof digital ticket that proves who logged in and what role they have.
MCP (Model Context Protocol)	A shared standard for how AI agents call outside tools/systems.
Human-in-the-loop (HITL)	A safety pattern where a human must approve a sensitive action before it takes effect.
Serverless	Cloud code that runs on demand without a dedicated server to maintain.
Guardrail	An automatic safety filter checking AI input/output for harmful or manipulative content.
Infrastructure as code	Describing cloud setup in files so it can be recreated and reviewed like software.